ChatGPT

Executive Summary

This plan breaks down the cpp00 exercises into small steps with time estimates, links each step to prerequisite C++ concepts, and recommends resources for quick study. It emphasises iterative implementation with testing at every step, highlights common pitfalls in beginner C++/OOP, and includes reflection activities to reinforce learning. A detailed schedule (with buffer time) and a deliverables checklist ensure steady progress. Recommended tools (editor/IDE, compiler flags, build scripts) and study references are tabulated for easy comparison. Key references from authoritative sources (e.g. C++ documentation and tutorials) are cited throughout to guide the learning process.

Task Breakdown (Exercises and Time Estimates)

- **Prep & Setup (30 min):** Read all exercise descriptions carefully; set up project directories/ex00, ex01, ex02 with stub files and Makefile.
- **Exercise 00 – Megaphone (1.5 hrs):**
- *Interpret requirements (10 min)*: Understand command-line input handling and output format.
- *Implement logic (50 min)*: Read `argv` strings, convert each character to uppercase using `std::toupper` 【24†L242-L249】, and output results. Handle no-argument case ("LOUD... NOISE").
- *Test & Refine (30 min)*: Verify with sample inputs (multiple arguments, special characters, no args). Check all edge cases (empty string, mixed case).
- **Exercise 01 – PhoneBook (4 hrs):**
- *Design classes (30 min)*: Outline `Contact` (fields for first/last name, nickname, phone, secret) and `PhoneBook` (array of up to 8 contacts). Plan public/private members following encapsulation.
- *Implement* `Contact` *(45 min)*: Code member functions for input and display (get/set), ensuring private data and public methods.
- *Implement* `PhoneBook::add()` *(45 min)*: Add contacts in a fixed-size array (max 8); on 9th, overwrite oldest. Use no dynamic allocation.
- *Implement* `PhoneBook::search()` *(1 hr)*: Format and display contacts in table (4 columns: index, first, last, nick; each 10 chars wide). Use `<iomanip>` manipulators (e.g. `std::setw(10)`, right alignment)【8†L539-L542】. Truncate long strings with a dot at the end.
- *Interactive loop and edge cases (30 min)*: Build command loop (`ADD` , `SEARCH` , `EXIT`). Ensure input fields cannot be empty. Handle invalid commands/indices.
- *Test & Debug (30 min)*: Manually try adding >8 contacts, search by valid/invalid index, format output. Check for off-by-one errors.
- **Exercise 02 – Account (2.5 hrs):**
- *Understand assignment (15 min)*: Read `Account.hpp` and `tests.cpp` (if available) to infer missing implementation. Identify required static members (e.g. account counters, totals) and methods.
- *Implement* `Account` *class (1 hr 15 min)*: Write `Account.cpp` with constructor/destructor, static data members and functions (e.g. tracking total accounts, deposits, withdrawals). Use only C++98 features (as per general rules) and static members for shared state【14†L268-L276】.
- *Run tests and fix (45 min)*: Compile with provided `tests.cpp`; adjust implementation so output matches reference log exactly (ignoring timestamps). Verify constructor/destructor messages (order may vary).

- **Wrap-Up (45 min):** Update code formatting; review that each exercise's `main` executes correctly. Reflect on lessons learned and update study notes (see below).

Total estimated time: ~8 hours (1 day), including short breaks and a contingency buffer.

Prerequisite Concepts & Language Features

- **Command-line arguments & strings (Ex00):** Know `int main(int argc, char** argv)`, use of `std::string` or C-string iteration. Familiarity with character functions (`std::toupper` from `<cctype>` 【24†L242-L249】).
- **Loops and I/O streams:** Using loops (`for` or `while`) to process each argument; writing output to `std::cout` with newline.
- **Classes and encapsulation (Ex01 & Ex02):** Understanding of defining classes (`class Name { ... };`), public vs private members. Encapsulation of data (fields private, methods public) 【18†L166-L174】. Constructors, destructors, and method definitions (in .cpp vs .hpp).
- **Fixed-size arrays:** Declaring a fixed array of `Contact` objects (`Contact contacts[8];` in C++98). Indexing from 0; managing overwrite of oldest contact (e.g. using modulo or shifting).
- **Static class members (Ex02):** Use `static` keyword inside class for members shared across all instances (e.g. a global count)【14†L268-L276】. Initialize static members outside class definition.
- **Formatting output (Ex01):** `<iomanip>` manipulators: `std::setw(int)`, `std::right`, `std::setfill(char)` 【8†L539-L542】. Truncating strings (use `std::string::size()` or substr).
- **User input validation:** Checking `getline(std::cin, ...)` input; handling empty lines or wrong commands.
- **Standard Library restrictions:** Per general rules, avoid dynamic memory (new/free) and advanced STL (vectors, etc.) until allowed. Use basic C++98 only.

Study Resources (recommended, English)

Resource	Focus / Notes
cppreference.com	Authoritative reference for C++ (string, IO, manipulators, etc.). E.g. see `<iomanip>` functions【8†L539-L542】, `std::toupper` in `<cctype>` 【24†L242-L249】, and `static` members【14†L268-L276】.
LearnCpp.com	In-depth C++ tutorials (OOP, classes, arrays, std::string, I/O). Good for beginners. Chapters on **Classes and OOP** and **Basic IO** are directly relevant.
cplusplus.com	C++ tutorials and reference (classes, <string>, <iostream>, <iomanip>). Useful examples for beginners (e.g. formatting with `setw`).
C++ Primer (Lippman)	A comprehensive book (if available) on C++ fundamentals and OOP concepts. Useful for deeper study of classes, static members, and good coding practices.
StackOverflow / Forums	For quick answers on specific issues. Use to search common errors (e.g. "phonebook class contact" or array bounds).

Resource	Focus / Notes
Module Instructions/ Videos (if provided)	The exercise intranet often has hints (e.g. iomanip usage) – consult any official instructions or tutorial videos.

Testing & Debugging Strategies

- **Incremental compilation:** After each small change (e.g. implementing a single method), compile with `-Wall -Wextra -Werror` to catch warnings as errors. Fix issues before moving on.
- **Unit tests for Ex00:** Try example commands given in spec. Test edge cases: no arguments, single character, mixed-case and special characters. Confirm output exactly matches expected (case and punctuation).
- **Interactive testing for Ex01:** Manually add 1–8 contacts, display them, and then add a 9th to ensure the oldest is replaced. Check column formatting: run long names to ensure truncation with a dot (.) at 10th character. Use both valid and invalid inputs (non-numeric index, out-of-range index) for SEARCH. Verify program continues or exits correctly.
- **Use provided tests for Ex02:** Compile `tests.cpp` with your `Account.cpp`. The program should run without errors and match the log output (except timestamps). If errors occur, re-examine constructors, static initializers, and arithmetic (e.g. sum updates).
- **Debug aids:** Insert temporary `std::cerr` prints to trace flow (e.g. index values) if logic seems wrong. Use a debugger or `printf`-style prints for interactive parts. For memory issues (though dynamic memory isn't used), tools like Valgrind can check for leaks or invalid accesses (optional).
- **Peer-review/ Rubber-ducking:** Explain code logic to a peer or an imaginary "rubber duck" to find logic flaws.

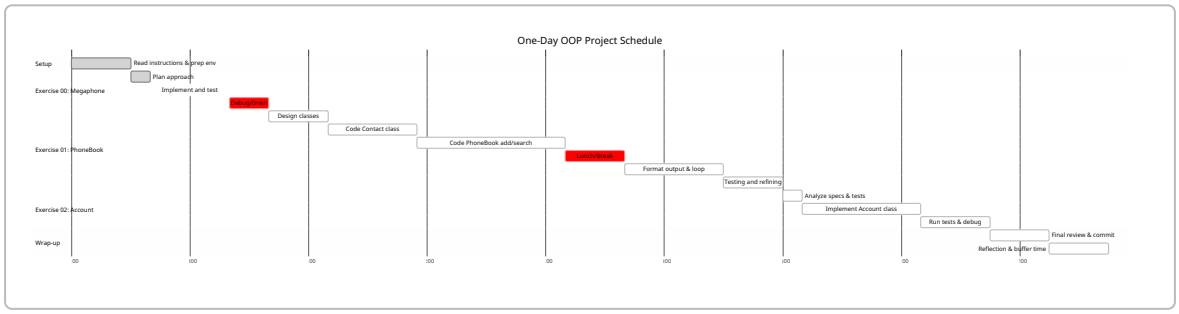
Common Pitfalls & Anti-patterns

- **Off-by-one errors:** In loops (e.g. indexing 0–7 vs 1–8 in phonebook), carefully manage indices (especially when wrapping at 8).
- **String handling:** Forgetting to check for empty input (empty names should be rejected). When truncating names, don't forget to add a dot if the string length exceeds the column width.
- **Static member initialization:** Declaring `static int X;` inside class and also defining/ initializing it properly in `Account.cpp` (outside class). Missing this causes link errors.
- **Formatting mistakes:** Not resetting fill character after using `setfill`, or misplacing `std::right`. Each `setw` only applies to the next output field. Ensure pipes `|` are placed correctly between fields.
- **Forgetting `return 0` in `main`** (though C++98 allows omission). But ensure all code paths either exit or loop correctly.
- **Misusing `std::toupper`:** As documented, cast characters to `unsigned char` before `std::toupper` 【24†L270-L279】 if working with `char`. Otherwise undefined behavior may occur on negative `char` values.
- **Too-large classes:** Keep `Contact` and `PhoneBook` design simple. Don't embed unnecessary logic in one class; each class should have a clear single responsibility.
- **Ignoring project rules:** The module prohibits certain functions (e.g. `printf`, STL containers). Using forbidden functions will incur a 0 grade. Strictly use allowed C++ features.

Reflection & Consolidation Activities

- **Summarize in your own words:** After each exercise, write down what you did and why. For example, note how you used an initialization list in a constructor or why you used `static`. Putting this into a personal journal or comments helps reinforce learning.
- **Create flashcards:** Use an app or index cards for new concepts (e.g. "What does `std::setw(10)` do?", "What is a static class member?"). Review them periodically.
- **Extend the mini-project:** Add a small feature to each exercise to apply concepts. E.g., for the PhoneBook, allow editing or deleting a contact. For Megaphone, support lowercase toggling. This deepens understanding.
- **Peer teaching:** Explain one concept (like how you managed the 8-contact array) to a study partner or online forum (StackOverflow, Reddit). Teaching is one of the best ways to consolidate.
- **Code Review:** After finishing, review your code with a checklist (no unused variables, consistent naming, comments where needed). Identify one thing you would improve.

Schedule & Timeline



This timeline totals roughly 8 hours, with breaks and a half-hour buffer at the end. Adjust as needed based on individual pace.

Deliverables Checklist & Success Criteria

- **Exercise 00 (Megaphone):**
- Files: `megaphone.cpp`, `Makefile` (or integrated build).
- **Success:** Program compiles cleanly. Running `./megaphone arg1 arg2 ...` prints all args in uppercase (one space-separated line). With no args, prints exactly `* LOUD AND UNBEARABLE FEEDBACK NOISE *`. Matches examples.
- **Exercise 01 (PhoneBook):**
- Files: `PhoneBook.hpp/cpp`, `Contact.hpp/cpp`, `Makefile`.
- **Success:** Running the phonebook executable allows commands `ADD`, `SEARCH`, `EXIT`. `ADD` collects 5 non-empty fields, stored correctly. After 8 contacts, the 9th overwrites the oldest. `SEARCH` prints a neatly formatted table (4 columns, width=10, right-aligned with dots for overflow). Selecting a valid index displays full contact info; invalid index is handled gracefully. No crashes on edge cases.
- **Exercise 02 (Account):**
- Files: `Account.hpp` (provided), `Account.cpp` (recreated), `tests.cpp`, `Makefile`.
- **Success:** Compiling with `tests.cpp` passes without errors. Running it reproduces the output in `tests.cpp`'s reference log exactly (apart from timestamps). All static counters (number of accounts, total balance, etc.) update correctly. Destructor messages appear (order not critical).

- **General:** Code style clean (consistent indentation, meaningful names). No forbidden functions or namespaces used. All output ends with newline as required. Code version-controlled (e.g. commit after each exercise).

Tools, Editor/IDE, Compiler Flags, Automation

Tool/Component	Purpose/Configuration
Compiler: g++ / clang++	Use C++98 standard (e.g. `-std=c++98`). Enable warnings as errors: `-Wall -Wextra -Werror`. Optionally `-pedantic` for strict ISO compliance.
Build Automation: Make	Write a simple `Makefile` (one target per exercise). Use it to compile `.cpp` files and link. For example, `make ex00`, etc.
Editor/IDE: VSCode (with C++ extension), CLion, Code::Blocks, or similar.	Any editor with syntax highlighting and IntelliSense for C++ helps spot typos. Configure to use C++98 and show compiler warnings.
Debugging Tools: GDB (GNU Debugger), Valgrind (for leaks, if needed).	Use GDB to step through logic. Valgrind to check memory (mostly relevant if using pointers, but still good practice).
Version Control: Git (with GitHub/ GitLab).	Regular commits (after each exercise or major milestone). Helps track changes and revert if needed.
Static Analysis: `cppcheck` or clang-tidy (optional).	Run on your code to catch subtle issues (unused variables, possible mistakes).
Output Comparison: `diff` or `cmp`.	To compare your program's output to expected (especially for Exercise 02). Useful in debugging final output mismatches.

For the editor, set indentation style (4 spaces), line-length ~80, and enable "format on save" if available. In the IDE/compiler settings, treat warnings as errors. Ensure the `Makefile` runs the compiler with these flags automatically.

By following this structured plan, the developer can implement all cpp00 exercises efficiently in one day, with a clear understanding of each concept and a set of resources to clarify any doubts. Each exercise's success criteria and testing strategy ensure that the code works correctly, and the reflection activities help solidify the newly gained OOP knowledge for future projects.

Sources: Authoritative C++ references were used to guide this plan (e.g. standard library documentation【8†L539-L542】【24†L242-L249】, C++ language feature descriptions【14†L268-L276】, and well-regarded tutorials). These sources provide detailed explanations of the concepts and functions mentioned above.
